

CS 440/540 Program #4: Rusty to x86-64

Due date on Canvas

For this last programming assignment you are going to generate x86-64 code to implement the semantics of the input Rusty code. I talked about x86-64 in class and there are examples online; however, in this handout I will summarize some of the most important information. For this assignment, we are going to make lots of simplifying assumptions - be sure to read this document carefully! I strongly suggest you deal with the different issues in the order I discuss them below. For each, test completely, including compiling and running on zeus to see if what you generate is correct (both syntactically and in terms of the underlying computation). Examples of each functionality are available online.

One assumption you can make for this assignment is that all input that you will be tested on will be correct with respect to our type system. It is ok if your program crashes on bad input (i.e. type incorrect input).

I strongly suggest you work on this assignment incrementally, testing each added part thoroughly before moving to the next. Your assignment will be graded on how far down the following list (which roughly matches this document order) you are able to generate *correct, running* code:

Functionality correctly implemented	CS440 students	CS540 students
Output (print, println) - REQUIRED	10%	10%
Arithmetic expressions involving constants	10%	10%
Relational and boolean operators	5%	5%
Procedure call control flow	15%	10%
Procedure call data flow (parameters)	15%	10%
Local variables and scopes	20%	20%
Input	5%	5%
While/if (including nesting)	20%	20%
Arrays	2% (extra credit)	10%

The job of your compiler is to produce a .s file with a main program called 'rusty_main' (like the example in the next section). I am providing a rusty_main.c which contains some definitions and calls a function 'rusty_main()' - this code will be compiled with your .s file to create an executable. I set it up this way to make your task easier since the code needed in a C main is a little different from that in a regular function. To build a runnable program from rusty input,

```
your_parser < rusty_input > file.s
gcc rusty_main.c file.s
```

The first line is used to create a .s file from the rusty input. Then this .s file is compiled with the given main program to create an executable (a.out) file. Now we can run it!!

Output: io1.r

Implementing the print (and println) function first will make it easier to test your code and will have you generating the setup/finish code that will be used by every program you generate. We are going to implement print functions using a C printf. The format of a C printf call for an integer is printf("%d ", the_int);. For the rusty program 'fn main () { print(42); }', the following output when compiled with the rusty_main.c results in an executable that prints 42.

```

.text
.globl rusty_main
.type    rusty_main, @function
rusty_main:
    movl $42, %r10d
    movl %r10d, %esi           # parameter 2 in %rsi/esi
    movq S1(%rip), %rdi       # parameter 1 in %rdi/edi - S1 is the string "%d "
    movl $0, %eax
    call printf
    ret
.size    rusty_main, .-rusty_main
.section .note.GNU-stack,"",@progbits

```

The parts of the code that implement the print function are in red. To generate the code above, the `expr --> INTEGER` production will generate the load of constant 42 into `%rd10d`. When processing the write function, this register is copied into the second parameter register (`%esi`) and a string pointer `S1` defined in `rusty_main.c` is the first parameter (`%edi`). This code is generated by the `expr --> PRINT (expr)` production. NOTE: The output from my compiler, included in the code samples is actually much longer because it is doing register save/restore work that is needed in general. However, that save/restore code isn't really needed for this simple program.

Input is a trickier and will be addressed after we talk about variables.

Arithmetic Expressions involving constants: `exprs.r`

Next, work on generating x86-64 code for expressions that use constant values. At the leaves of tree, you need to put the constant values into registers (`movl` instructions). You need to have an attribute for some of the non-terminal symbols (like `expr`) that keeps track of what register is holding the relevant value. The leftmost column of the table below show x86-64 instructions that implement the Rusty statement `print (5*3) + (8*2);`.

x86-64	Registers in use after statement	Registers Available after statement
<code>movl \$5, %r10d</code>	{%r10d}	{%r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl \$3, %r11d</code>	{%r10d, %r11d}	{%ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>imull %r11d, %r10d</code>	{%r10d}	{%r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl \$8, %r11d</code>	{%r10d, %r11d}	{%ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl \$2, %ebx</code>	{%r10d, %r11d, %ebx}	{%ebp, %r12d, %r13d, %r14d, %r15d}
<code>imull %ebx, %r11d</code>	{%r10d, %r11d}	{%ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>subl %r11d, %r10d</code>	{%r10d}	{%r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl %r10d, %esi</code>	{}	{%r10d, %r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl S1(%rip), %edi</code>	{}	{%r10d, %r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>movl \$0, %eax</code>	{}	{%r10d, %r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}
<code>call printf</code>	{}	{%r10d, %r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}

As discussed in class, the above code uses registers to hold results temporarily. Because the number of registers is bounded, you need to keep track of which ones you are actually using at any given point in time. You have 15 registers available but it is safest to only use a subset, saving the other registers like `%eax` and the parameter registers for special use. The subset of 32 bit registers I use is {%r10d, %r11d, %ebx, %ebp, %r12d, %r13d, %r14d, %r15d}. Assuming you "allocate" a register when you need one and "free" a register when you are done with it, this is plenty for any example that I will come up with. You need a data structure that tells you what registers are available (initially, all registers are available). Write two functions: one that gets an available register and another that returns a register that you are no longer using. The second and third columns of the above table show what registers are in use and free after each instruction. Note that we are using long instructions (`movl`, `addl`, ...) because integers and booleans will be stored as long values in the 32-bit portion of our registers. Getting this working for constants means that if you deal with variables by loading them into registers, all of work of computing expressions with identifiers is already done.

Don't worry about generating efficient code. That is much more difficult and doesn't fix anything a good optimizer wouldn't fix. Also, don't worry about generating code identical to the examples online or in this document. As long

as the computation will produce the same result, your answer is fine. Most of the computations should be obvious. Booleans should be implemented as the integers 0 and 1.

Hint: To get this right, you need to look for rules where information in multiple registers is used to compute something where the result is in one register. For example: `exp : exp + exp` you need to be sure to free a register because your generated code will use the contents of two registers, leaving the result in one of them. The other register is no longer needed. Many different Rusty statements use several registers; these also must be freed when no longer in use.

Procedure Call/Return control flow: fun1.r

Implementing the control flow for procedure call return is relatively straightforward assuming you have no parameters. Remember in class, I talked about the four locations where you need to add code for functions. Below, I have given you what I am generating to handle basic control flow and save registers.

To test, I typically move my code from constant expression evaluation into a parameter-less function and then call it. Use 'call' and 'ret' to manage control flow. This is also a good time to add code to save and restore registers (using the stack) and build an activation record for the calls. For every function, I add the code below that deals with the callee-save 64-bit registers. I'm choosing an additional frame size (128 bytes) because I will need to have space for the 8 caller-save registers plus space (upto to 64 bytes) of local variables. You will not be tested on programs where you can't fit into this frame size (assuming you implement as I describe here). You can make this number larger if you want, but smaller might be a bad idea. Control flow statements are marked in red and locations where data flow statements will go are in blue.

At the start:

```
pushq %rbx
pushq %rbp
pushq %r12
pushq %r13
pushq %r14
pushq %r15
subq $128, %rsp
```

At the return point:

```
#move the result to %eax
addq $128, %rsp    # my frame size
popq %r15
popq %r14
popq %r13
popq %r12
popq %rbp
popq %rbx
ret
```

At call points:

```
movq %rdi, 64(%rsp)
movq %rsi, 72(%rsp)
movq %rdx, 80(%rsp)
movq %rcx, 88(%rsp)
movq %r8, 96(%rsp)
movq %r9, 104(%rsp)
movq %r10, 112(%rsp)
movq %r11, 120(%rsp)
// load parameters
call function
movq 64(%rsp), %rdi
movq 72(%rsp), %rsi
movq 80(%rsp), %rdx
movq 88(%rsp), %rcx
movq 96(%rsp), %r8
movq 104(%rsp), %r9
movq 112(%rsp), %r10
movq 120(%rsp), %r11
//move eax to another register
```

We are using quad word operations (i.e. `movq`, `pushq`, `popq`) because we want to save all 64 bits of the relevant registers just in case.

The result of a function is always put into `%rax/%eax`. I usually add code to make this happen when I'm working on basic function calling. You can see where it goes in the return point code above. Since the caller expects results in

%eax, you just need to copy the correct register there.

Procedure Call/Return Data flow: fun2.r, fun3.r

Parameters are passed into our functions using registers: %rdi/edi, %rsi/esi, %rdx,edx, %rcx/%ecx, %r8/%r8d, %r9/%r9d. Additional parameters would go onto the stack - however you will only be tested on 5 or fewer parameters. You can see this pattern in the scanf and printf calls above. The integer parameters are passed by value.

Since we aren't using the parameter registers (%rdi/edi, %rsi/esi, %rdx,edx, ...) to do any of the work described above AND because we are saving (and restoring) the registers for calls, you can use the parameter registers directly in your generated function code like I do, rather than saving the parameters onto the runtime stack and using them from there (which also works but is slower and takes up space that I want you to use for local vars).

NOTE: This means that there is some code that really won't work easily, like the following:

```
fn g(a: i32, b: i32) {
    f(b, a);
}
```

I will not test you on code like what is above. This is because in the code on the left, if you keep formal parameters a and b in %rdi and %rsi respectively, you will get the wrong value for the second parameter in the function call because that first parameter in the call will overwrite a. I WILL NOT TEST YOU ON CODE LIKE THIS. To achieve this, you would need to write the code more like:

```
fn g(a: i32, b: i32) {
    let x = a;
    let y = b;
    f(y,x);
}
```

which effectively saves a and b locally (discussed next section). I might test you on something like this. Recursion generally requires local variables so you will want to do the next step before you worry about whether recursion will work.

Local variables and scope: vars1.r vars2.r fun4.r

All other local variables in a function should be saved on the runtime stack and will be accessed using D(%rsp) notation, where the D corresponds to that variable's offset into the frame. Local integers can be defined in let statements and are size 4 bytes. As mentioned previously, I always allocate 128 bytes for each stack frame. I use some of these bytes to save (caller-save) registers I will be using when needed. You can assume that there will be no more than 10 local variables in a function, so the frame is probably too large but better safe than sorry. In the given sample output, you will see references to local variables as offset(%rsp). For example, if I have two local integer variables x and y, x might live at 0(%rsp) and y might live at 4(%rsp).

You will need to use a symbol table just as with assignment 3; however, the important information is not the type, but information that will tell you whether every identifier is global, local or a parameter so that you can generate the correct code to for the given identifier.

For the example code below with parameters and local variables

```
fn f(x: i32, y: i32) {
    let a = 2;
    let b = x + y;
    print ((x + a) * b);
}
```

variables x and y would be found in %edi and %esi respectively, since they are parameters. Local variables a and b would live at 0(%rsp) and 4(%rsp) in my code. For example, if I wanted to assign the value of x to a, the assembly

code `movl %edi, 0(%rsp)` would accomplish that. Once this is implemented correctly, you can test your code on recursive functions.

You will also need to correctly handle code like the following function:

```
fn f(x: i32, y: i32) {
  let a = 2;
  { let a = x + y; print (a); };
  print (a);
}
```

In this code, there are two instances of `a`. If you look at what I generate, the first `a` will live at `0(%rsp)` and the second at `4(%rsp)`. The trick is to figure out how to do this.

Input: io2.r

Code for input is given in a number of different samples. For some reason (which I haven't had time to debug), calling the `scanf` function isn't working. Because I do want to be able to do things like initialize arrays, I've modified my input to use a random number generator so that I can get interesting input to my functions. To get this input, you will generate code to call `'rusty_input()'`, which takes no parameters. The result of this function will be in `%eax`.

Control Flow: if1.r if2.r if3.r while1.r while2.r

You need to generate the appropriate labels and branching for the control flow statements (`if/while`) of Rusty. To do this you will need to first add code to evaluate relational expressions. This involves using `cmp` statements to set a register to either 0 or 1. For example, `( x < y )` (assuming `x` and `y` are in `%edx` and `%ecx` respectively and assuming I want to put the result into `%edx`) would look like:

```
    cmp    %ecx, %edx           # note the order is "reversed"
    setl   %al                 #uses condition bits to figure out the result of < to put into %al
    movzbl %al, %edx           # copies %al to %edx zeroing out the top parts
```

Note that it is the `set` instruction that would change depending on the relational operator. Boolean expressions themselves are treated like the arithmetic operators.

The `IF` and `WHILE` statements will need to use labels to direct the control flow based on the result of the conditional. For example, consider `if exp1 { exp2 } else {exp3}`. If we assume that the result of `exp1` is in `%ecx`, you would want to generate:

```
    cmp    $0, %ecx           # value true or false?
    je     LABEL1
    # code to evaluate exp2 goes here
    jmp    LABEL2
LABEL1:
    # code to evaluate exp3 goes here
LABEL2:
```

This means you will have to have actions inside the production for `IF` (and `while`) statements. An additional complexity of an `IF` statement that the statement returns a value. For example, the statement `let x = if (> y 0) {1} else { 2}` means that `x` will get the value 1 if `y > 0` and 2 otherwise. You will need to be sure that in the generated code, the result is always in the same register no matter which branch of the `if` statement was taken.

One way to deal with the fact that control flow statements can be nested is to use the YACC stack to keep track of the labels associated with each loop or `if` statement by associating them with one of the terminal symbols of the production. An alternative implementation is to use a stack onto which you push active labels. Once you finish with the particular statement, pop the associated labels. Remember that control flow statements use registers and they need to be freed once you are done with them.

Arrays

You will be implementing 1-D integer arrays. Arrays are created globally before any function definition and then can be used as global variables. I have some sample code that I generate that you can use as a guide. If you got this far, I have confidence that you can get this last step implemented!

Submitting

Submit on Canvas as a tar or zip file. Be sure to include a Makefile OR explicit directions on how to build your submission. Be sure to check that you are submitting what you intend! There is not time to fix mistakes in your submissions.